LINFO 1121

DATA STRUCTURES AND ALGORITHMS

UCLouvain

TP4: Dictionnaires

Question 4.1.1 Implementation of a map

Any ideas?

Unordered list, ordered list, BST, array-based mapping, ...

Question 4.1.2 a + b = x

Given a set S of size n, a number x: find if there exists a pair of element a,b from S such that a + b = x

We have seen that we can solve this problem by first sorting the elements.

This method is in O(n log(n)).

Can we do better ?

Question 4.1.2 a + b = x

Given a set S of size n, a number x: find if there exists a pair of element a,b from S

such that a + b = x

We can notice that if such pair exist, then we have a = x - b

For each element v of S, we must find x - v

x = 46

1, 40, 18, 34, 23, 55, 12

X = {1, 40, 18, 34, 23, 55, 12}

Is 46 - 1 = 45 in the set X ?

Is 46 - 34 = 12 in the set X ?

O(n + n) time complexity ! Downsides ?

Question 4.1.3 Modulos

$$(a + b) \,\%\, M \qquad ((a \,\%\, M) + b) \,\%\, M$$

Prove that these two values are equal

$$a = (c \cdot M) + (a \,\%\, M) \qquad \text{où } c = \lfloor \frac{a}{M} \rfloor$$

$$\begin{aligned}(a + b) \,\%\, M &= ((c \cdot M) + (a \,\%\, M) + b) \,\%\, M \\ &= ((a \,\%\, M) + b) \,\%\, M\end{aligned}$$

Question 4.1.3 Java hash for String

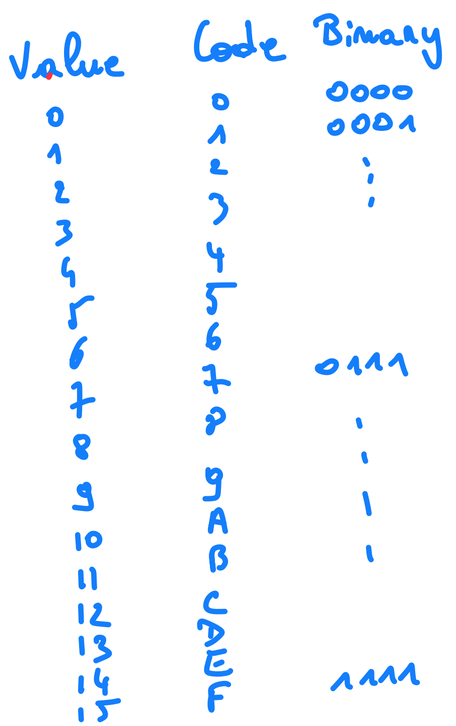
Java hash formula for String: $$(\sum_{i=1}^{n} R^{n-i} s_i) \% M$$

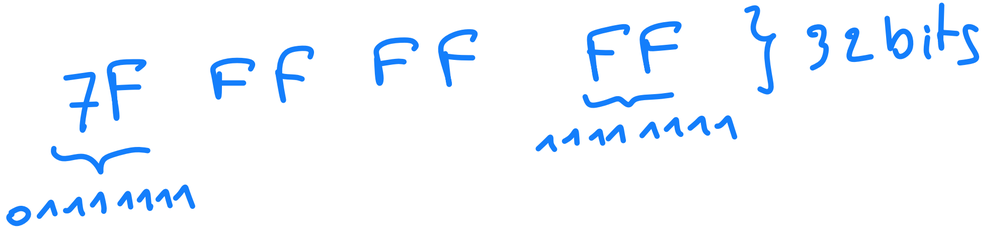
Complexity is ~ n

But, Java uses a cache. So computing the hash N times requires only n + N operations

Question 4.1.4 Générer une clé a partir d'un hash

Private int hash(Key x) { return (x.hashCode() & 0x7fffffff) % M);





Question subsidière: x & 0x7FFFFFFF = abs(x) ?

X Bits	Signed value (Two's complement)	X & F	Signed value (Two's complement) of X&F
0000	0	000	0
0001	1	0001	1
0010	2	0010	2
0011	3	0011	3
1100	-4	0100	4
1101	-3	0101	5
1110	-2	0110	6
1111	-1	0111	7

Question 4.1.5 types de tables de hachages, map et SETS

Nom	Type	Methode	Notes
HashMap	Dictionnaire	Chaining (+ Tree)	
HashTable	Dictionnaire	Chaining	Threadsafe
ConcurrentHashMap	Dictionnaire	Chaining	Threadsafe
HashSet	Set	Chaining + Tree	
ConcurrentHashSet	Set	Chaining	Threadsafe
LinkedHashMap	Dictionnaire	Chaining	Maintien l'ordre d'insertion
IdentityHashMap	Dictionnaire	Chaining	Vérifie les clés par références
WeakHashMap	Dictionnaire	Chaining	Weak keys
TreeSet	Set	Tree	
TreeMap	Dictionnaire	Tree	

Question 4.1.6 COLLISIONS

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8			

What happens when 19 is added to the set ? (inserted at index 7)

Question 4.1.6 COLLISIONS

Original

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8			

Separate chaining

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1/19	8			

Linear probing

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8	19		

Question 4.1.6 COLLISIONS

Separate chaining

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1/19	8			

Linear probing

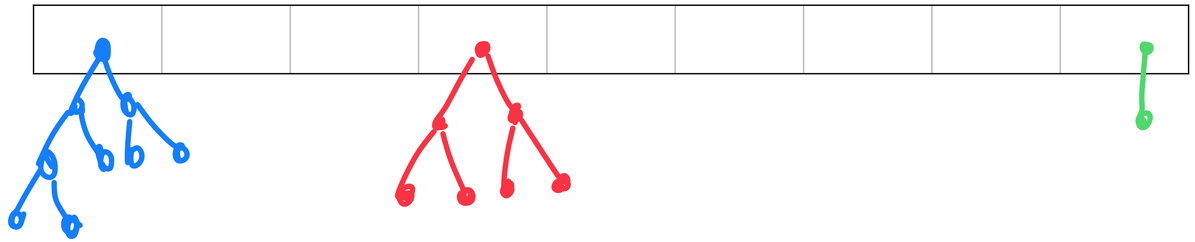
Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8	19		

Influence on run time ?

In which case use one or the other ?

Linear Probing vs Separate Chaining

HashMap use separate chaining and a tree in case of large number of collisions. This prevent the O(n) get/put induced with large lists.



Linear probing can not do that, a bad hash function will cause big cluster and you must pay the O(n) get/put.

Question 4.1.7 LOAD FACTOR

load factor = number of entry in the hashmap / capacité

For separate chaining, this is also the the mean number of entry per list

In linear probing, this is the proportion of filled entry in the array

Question 4.1.8 : Keys Iterator

Let’s look how java.util.Hashtable handle this !

Question 4.1.9 Linear probing avec marqueur

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8			

19 goes into bucket 8, but with linear probing it is placed in the next available spot

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8	19		

Question 4.1.9 Linear probing avec marqueur

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8	19		

If 8 is deleted, classical linear probing gives us this

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	19			

Question 4.1.9 Linear probing avec marqueur

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1	8	19		

The modified linear probing gives this result

Hash	0	1	2	3	4	5	6	7	8	9	10	11
Value			14		28	-3		-1		19		

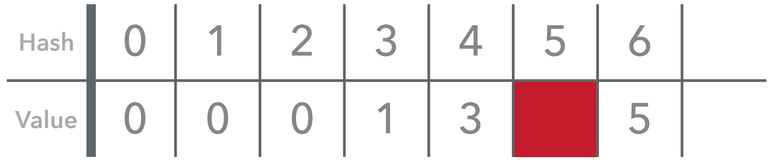
Better ? Worse? Pros/cons ?

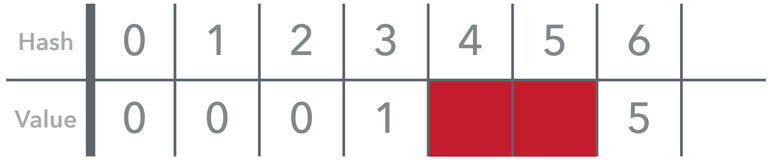
Question 4.1.9 Linear probing avec marqueur

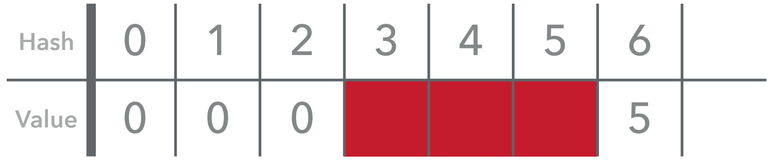
Two problems:

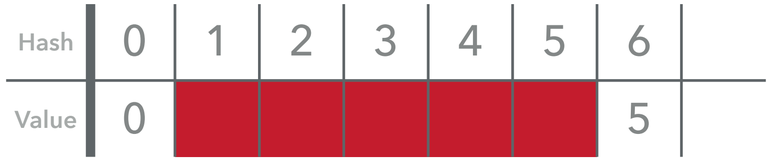
- Nothing removes the markers, gets and puts are slower

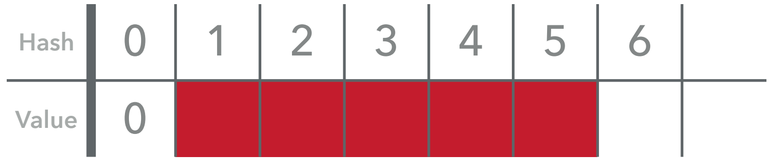
- As a general rules, one does more get/puts than deletes

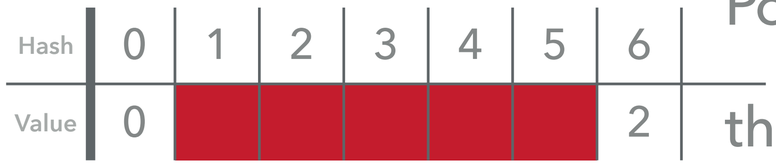












Possible to do "smarts" put, or to re-order the arrays, but the methods become complex

Question 4.9.10/11 : Rabin-Karp

You want to find a pattern of size *m* in a string of size *n*

The naïve way :

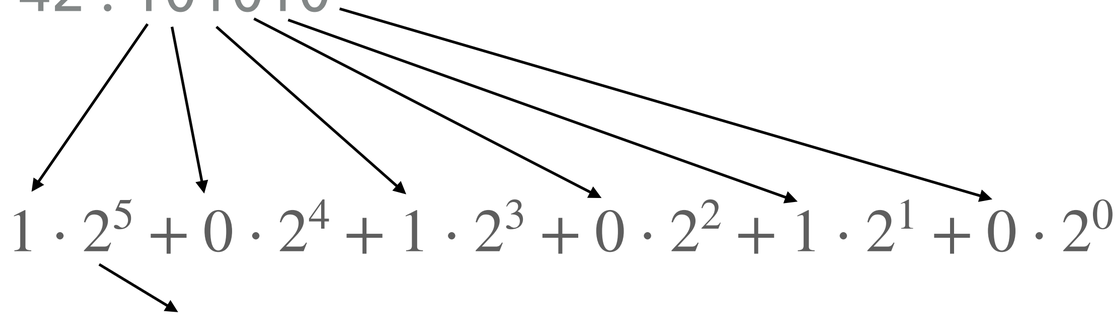
Of course you don't have to compare the entire pattern as you can return false as soon as one char is different

-> But the complexity is still O(*nm)*

Question 4.9.10/11 : Rabin-Karp

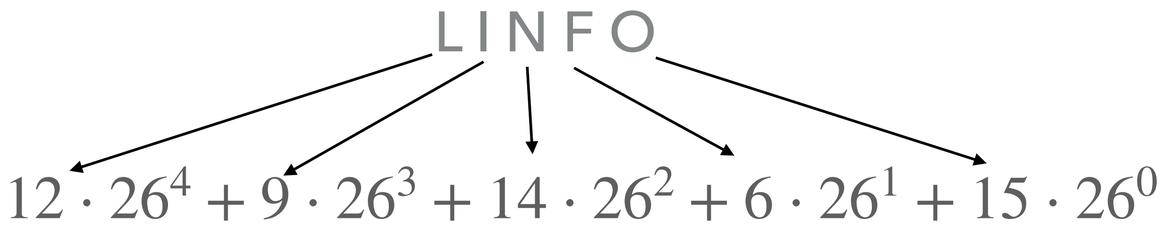
Comparing the hash of the pattern with the one of the considered portion of the string allows to do only one comparison -> how ?

Binary representation of 42 : 101010



Because we are in base 2

A letter is in base 26 :



Question 4.9.10/11 : Rabin-Karp

$$(t_i \cdot R^{M-1} + t_{i+1} \cdot R^{M-2} + \ldots \pm t_{i+M-1} \cdot R^0) \,\%\, Q$$

But we still have to compute the hash of the entire considered portion of the string (O(*m*) operations)

Question 4.9.10/11 : Rabin-Karp

$$s = t_0 t_1 \ldots t_n$$

$$x_i = t_i R^{M-1} + t_{i+1} R^{M-2} + \ldots + t_{i+M-1} R^0$$

$$\begin{aligned} x_{i+1} &= t_{i+1} R^{M-1} + \ldots + t_{i+M-1} R^1 + t_{i+M} R^0 \\ &= x_i R - t_i R^M + t_{i+M} = (x_i - t_i R^{M-1}) R + t_{i+M} \end{aligned}$$

But ! $h(x_i) = x_i \,\%\, Q$

$$h(x_{i+1}) = (((h(x_i) + Q - t_i R^{M-1} \,\%\, Q) \,\%\, Q) R + t_{i+M}) \,\%\, Q$$

Question 4.1.12

$$h([v_0 \cdots v_{n-1}]) = \sum_{i=0}^{n-1} v_i R^{(n-i-1)} \,\%\, M \qquad \left.\begin{array}{l} R = 32 \\ M = 1024 \end{array}\right\} \rightarrow M = R^2$$

$$= (v_0 R^{n-1} + \cdots + v_{n-1} R^3 + v_{n-3} R^2 + v_{n-2} R^1 + v_{n-1} R^0) \,\%\, M$$

$$= (\underbrace{\cdots}_{=0} + \underbrace{\cdots}_{=0} + \underbrace{\cdots}_{=0} + v_{n-2} R^1 \,\%\, M + v_{n-1} R^0) \,\%\, M$$

Seul les 2 derniers bits seront pris en compte

Et avec $R = 3$ et $M = 81$?

Question 4.1.13

```
// Monte Carlo
  private boolean check(int i) {
      return true;
  }


// Returns the index of the first occurrence of the pattern string in the text string.
  public int search(String txt) {
      int n = txt.length();
      if (n < m) return n;
      long txtHash = hash(txt, m);

      // check for match at offset 0
      if ((patHash == txtHash) && check(txt, 0))
          return 0;

      // check for hash match; if hash match, check for exact match
      for (int i = m; i < n; i++) {
          // Remove leading digit, add trailing digit, check for match.
          txtHash = (txtHash + q - RM*txt.charAt(i-m) % q) % q;
          txtHash = (txtHash*R + txt.charAt(i)) % q;

          // match
          int offset = i - m + 1;
          if ((patHash == txtHash) && check(offset))
              return offset;
      }

      // no match
      return n;
  }
```

Complexity ?

What happens if two different strings have the same hash ?

What can be done to solve this ?

MidTerms : Deserialization

2 4

It has two keys

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

min< 2 < max

min< 4 < max

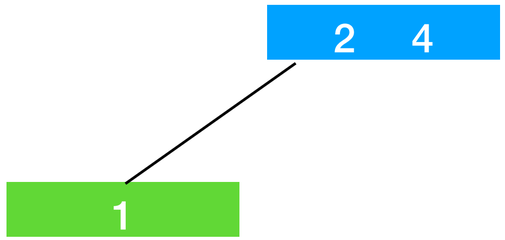
i = 0

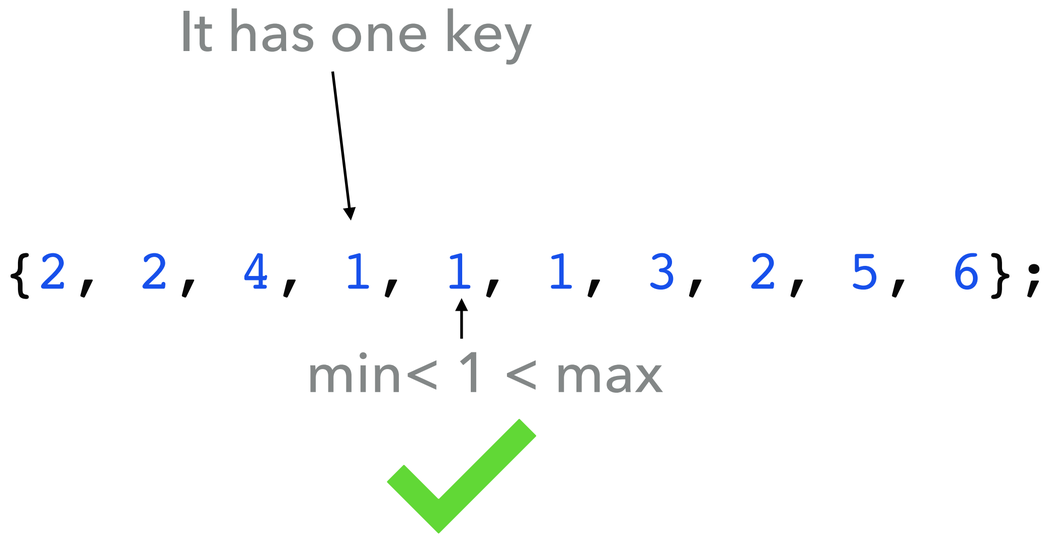
min = -∞

max = +∞

```
private Node preorderRead(Integer [] data, int i, int min, int max)
```

MidTerms : Deserialization





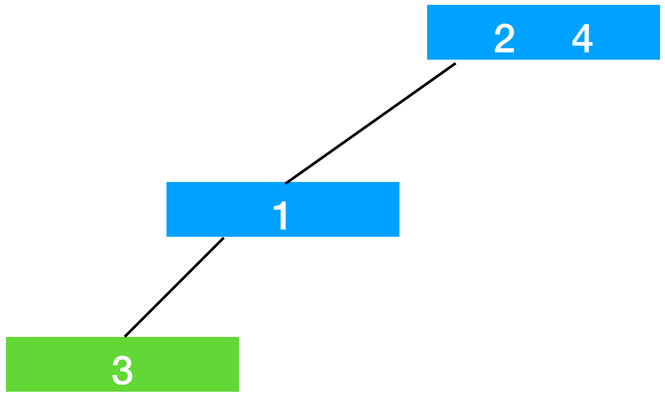
i = 3

min = -∞

max = 2

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization



It has one key

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

3 > max

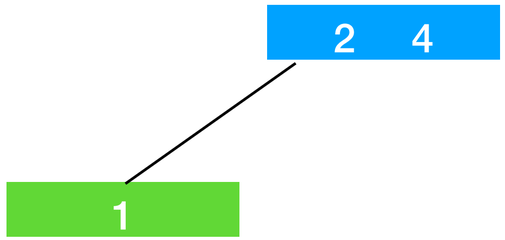
i = 5

min = -∞

max = 1

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization




```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

We finished the left branch, we should go on the right branch now

Which value i should take ?

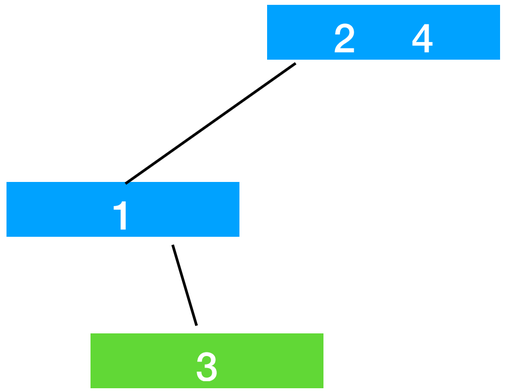
i = 3

min = -∞

max = 2

```
// Node of the 2-3 tree: remember: it can be a 2-node or a 3-node
static class Node {
    int[] keys = new int[3];     // Up to 3 during split
    Node[] children = new Node[4]; // Up to 4 children during split
    int n; // Number of keys actually in use (1 or 2, or 3 during split)
    int sizeNode = 1; // Number of nodes in the subtree
    int sizeKey = this.n; // Number of keys in the subtree
    boolean isLeaf() {
        return children[0] == null;
    }
}
```

MidTerms : Deserialization



It has one key

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

3 > max

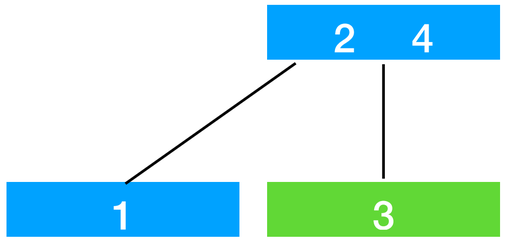
i = 5

min = 1

max = 2

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization



It has one key

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

min< 3 < max

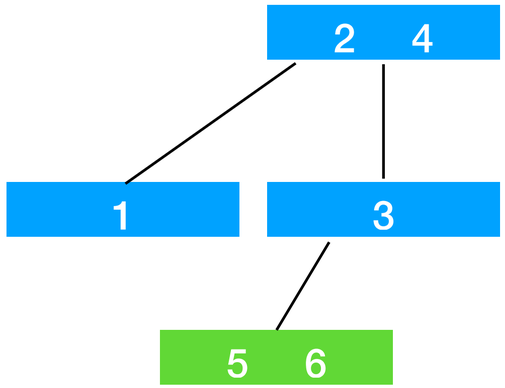
i = 5

min = 2

max = 4

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization



It has two keys

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

5 > max

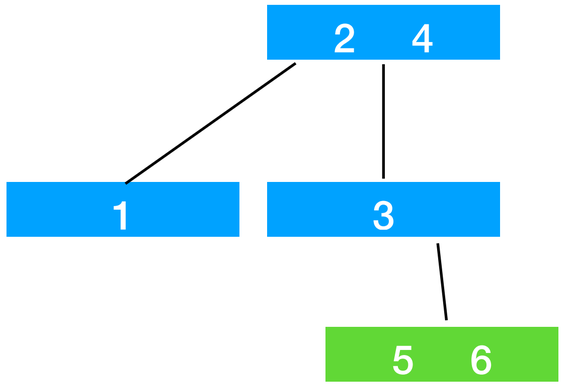
i = 7

min = 2

max = 3

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization



It has two keys

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

5 > max

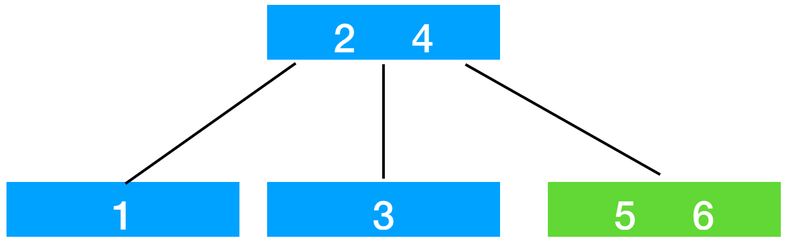
i = 7

min = 3

max = 4

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```

MidTerms : Deserialization



It has two keys

```
{2, 2, 4, 1, 1, 1, 3, 2, 5, 6};
```

min< 5 < max

min< 6 < max

i = 7

min = 4

max = +∞

```
private Node preorderRead(Integer [] data, int i, int min, int max) {
```